

王道数据结构笔记

第一章 绪论

1. 逻辑结构（抽象） -- 一对多 -- 存储结构（具体）
2. 计算 π 的算法有零个输入
3. 主定理: $T(n) = aT(n/b) + f(n)$
 - 若 $f(n) \neq n^{\log_b a}$, 则 $T(n) = \Theta(\max(f(n), n^{\log_b a}))$
 - 否则, $T(n) = \Theta(n^{\log_b a} \log n)$
 - **不等取大, 相等乘对数因子**

第二章 线性表

1. 线性表中: 数据元素 -- 一对多 -- 数据项
 - 数据元素就是线性表里存的元素
 - 数据元素是由数据项组成的
2. 单链表逆置方法: 摘下头结点, 从首元开始, 一次插入到头结点后面, 直到最后一个节点为止
3. 取链表中点方法: 设置快慢指针分别为 f 和 s, 初始时均指向首元, 之后 s 每次走一步, f 每次走两步。当 f 指向表尾时, s 正好指向链表中点
4. 有时候, 可以将单链表的变成循环单链表来加速一些循环操作的问题, 比如单链表循环位移

第三章 栈、队列和数组

1. 当 n 个不同元素入栈时, 出栈元素不同排列的个数为 $\frac{1}{n+1} C_{2n}^n$
 - 组合计算公式: $C_{2n}^n = \frac{A_{2n}^n}{n!}$
2. 一个存储单元 = 1 byte, 地址 1000H + 1byte = 1001H
 - 1000H + 1KB = 1400H
3. C 语言标识符只能以英文字母或下划线开头, 不能以数字开头
4. a b c d e 依次入栈, 在所有可能的出栈序列中, 以 d 开头的序列怎么求
 - d 之后入栈的只有 e
 - d 最先出栈, 下面的只能按照 c b a 的顺序出栈
 - e 有可能插入到 c b a 之间的任意位置出栈
 - 故
 - d e c b a
 - d c e b a
 - d c b e a
 - d c b a e
5. 循环队列首尾指针
 - 不管是队首指针指向首元的前驱还是队尾指针指向尾元的后继以牺牲一个单元来区分对空和队满, 以下结论均成立
 - 判断队满: $(rear + 1) \% SIZE == front$
 - 判断队空: $front == rear$ // TODO 似乎存在错误
 - 求元素个数: $(rear - front + SIZE) \% SIZE$

- 队尾指针指向首元的前驱，队首指针指向尾元：队尾指针需要初始化在数组末尾，即队尾指针 = $n - 1$
- 队首指针指向首元的前驱，队尾指针指向尾元：队首指针需要初始化在数组末尾，即队首指针 = $n - 1$

6. 带尾指针的循环单链表最适合做链式队列

- 尾删 $O(n)$ ，尾插、头插、头删 $O(1)$

7. 带头尾指针的单链表适合做链式队列

- 尾删 $O(n)$ ，尾插、头插、头删 $O(1)$

8.	名称	指针名	作用
	队头	front	弹出元素
	队尾	rear	插入元素

- 头删尾插

9. 使用带头尾指针的单链表做链式队列时，删除操作可能头尾指针都要修改。因为当队列只有一个元素时，需要修改尾指针

10. 三元组表存储稀疏矩阵无法随机访问

11. 三对角矩阵公式推导

1	第 $i - 1$ 行: $2 + (i - 2) * 3$ // 2是第一行的两个，后面都是3个一行
2	第 i 行: $j - i + 2$
3	故 $k = 2 + (i - 2) * 3 + j - i + 2 - 1 = 2i + j - 3$ // 下标从0开始
4	$k = 2 + (i - 2) * 3 + j - i + 2 = 2i + j - 2$ // 下标从1开始
5	对于下标从0开始的情况
6	$i = (k + 1) / 3 + 1$
7	$j = k - 2i + 3$

12. 矩阵与压缩数组下标换算公式

- 对称矩阵

矩阵 起点	数组 起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = \begin{cases} \frac{i(i+1)}{2} + j & i \geq j \\ \frac{j(j+1)}{2} + i & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, j = k - \frac{i(i+1)}{2}$
0	1	$k = \begin{cases} \frac{i(i+1)}{2} + j + 1 & i \geq j \\ \frac{j(j+1)}{2} + i + 1 & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor, j = (k - 1) - \frac{i(i+1)}{2}$
1	0	$k = \begin{cases} \frac{(i-1)i}{2} + j & i \geq j \\ \frac{(j-1)j}{2} + i & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor + 1, j = k - \frac{(i-1)i}{2}$
1	1	$k = \begin{cases} \frac{(i-1)i}{2} + j + 1 & i \geq j \\ \frac{(j-1)j}{2} + i + 1 & j > i \end{cases}$	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor + 1, j = (k - 1) - \frac{(i-1)i}{2}$

- 下三角矩阵

矩阵 起点	数组 起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = \frac{i(i+1)}{2} + j$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, j = k - \frac{i(i+1)}{2}$
0	1	$k = \frac{i(i+1)}{2} + j + 1$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor, j = (k-1) - \frac{i(i+1)}{2}$
1	0	$k = \frac{(i-1)i}{2} + j - 1$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor + 1, j = k - \frac{(i-1)i}{2} + 1$
1	1	$k = \frac{(i-1)i}{2} + j$ (当且仅当 $i \geq j$)	$i = \left\lfloor \frac{\sqrt{8(k-1)+1}-1}{2} \right\rfloor + 1, j = (k-1) - \frac{(i-1)i}{2} + 1$

◦ 上三角矩阵

矩阵 起点	数组 起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = \frac{n(n+1)}{2} - \frac{(n-i)(n-i+1)}{2} + j - i$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{n(n+1)}{2}-k)+1}-1}{2} \right\rfloor, j = k + i - \frac{n(n+1)}{2} + \frac{(n-i)(n-i+1)}{2}$
0	1	$k = \frac{n(n+1)}{2} - \frac{(n-i)(n-i+1)}{2} + j - i + 1$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{n(n+1)}{2}-(k-1))+1}-1}{2} \right\rfloor, j = (k-1) + i - \frac{n(n+1)}{2} + \frac{(n-i)(n-i+1)}{2}$
1	0	$k = \frac{(n-1)n}{2} - \frac{(n-i)(n-i+1)}{2} + j - i$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{(n-1)n}{2}-k)+1}-1}{2} \right\rfloor + 1, j = k + i - \frac{(n-1)n}{2} + \frac{(n-i)(n-i+1)}{2}$
1	1	$k = \frac{(n-1)n}{2} - \frac{(n-i)(n-i+1)}{2} + j - i + 1$ (当且仅当 $j \geq i$)	$i = n - \left\lfloor \frac{\sqrt{8(\frac{(n-1)n}{2}-(k-1))+1}-1}{2} \right\rfloor + 1, j = (k-1) + i - \frac{(n-1)n}{2} + \frac{(n-i)(n-i+1)}{2}$

◦ 三对角矩阵

矩阵 起点	数组 起点	矩阵 → 数组公式	数组 → 矩阵公式
0	0	$k = 3i + j - i$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \left\lfloor \frac{k}{3} \right\rfloor, j = k \% 3 + i - 1$
0	1	$k = 3i + j - i + 1$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \left\lfloor \frac{k-1}{3} \right\rfloor, j = (k-1) \% 3 + i - 1$
1	0	$k = 3(i-1) + j - (i-1)$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \left\lfloor \frac{k}{3} \right\rfloor + 1, j = k \% 3 + (i-1) - 1$

矩阵起点	数组起点	矩阵 → 数组公式	数组 → 矩阵公式
1	1	$k = 3(i - 1) + j - (i - 1) + 1$ (当且仅当 $j = i - 1, i, i + 1$)	$i = \lfloor \frac{k-1}{3} \rfloor + 1,$ $j = (k - 1) \% 3 + (i - 1) - 1$

- 行优先与列优先的区别为 i 与 j 互换
13. 二维矩阵 $A[i][j]$ 与 $A[i - k][j - k]$ 之间的内存距离为常数 $d = k * (m + 1) * L$
- k 为行和列的偏移量, m 为矩阵列数, L 为每个元素占用的内存大小
 - 下标从 0 开始、下标从 1 开始、矩阵行列不等时该公式均试用
14. 下标从 0 开始时, 下标值表示该元素前面的元素个数; 下标从 1 开始时, 下标值减 1 表示该元素前面的元素个数。

第四章 串

1. KMP 求 next 数组

- next 数组表示模式串中最长相等前后缀的长度 (对于一对相等的前后缀, 不一定是回文, 前后缀顺序相等)
- 若模式串下标从 0 开始: next 数组首元为 0, 数最长相等前后缀的长度即可
 - 注意, 这种只在代码里写
- 若模式串下标从 1 开始: 先求下标从 0 开始的 next 数组, 再将 next 数组整体加 1, 整体右移 1 位, 首元补 0
 - 此时模式串的下标为 1 和 2 的元素一定分别为 0 和 1
 - 这种情况考研常考
 - 考题中若模式串下标从 0 开始, 先按代码中的方法求下标从 0 开始的 next 数组, 然后整体右移 1 位, 首元补 -1
 - 考题中若看到 next 数组首元为 -1, 说明题设下标从 0 开始; 若首元为 0, 说明题设下标从 1 开始

2. KMP 求比较次数

- 失配了的那次比较也算

3. KMP 模式串最大滑动距离

- 不给主串, 一律认为模式串匹配到最后一个字符时失配并且该字符在模式串中不存在, 直接跳到模式串开头
- 最大距离 = 模式串长度 - 1

第五章 树与二叉树

树

- n : 树的节点数
- n_i : 度为 i 的节点数
- n_0 : 为 0 的节点个数/叶结点个数
- m : 树的度
- h : 树的高度

1. $n = \sum_1^m in_i + 1 = \sum_0^m n_i = n_0 + \sum_1^m n_i$
2. 对于一颗满 m 叉树, 第 i 层有 m^{i-1} 个节点, 到第 i 层为止共有 $\frac{m^i-1}{m-1}$
3. $n_0 = \sum_2^m (i-1)n_i + 1$
4. h 的取值范围为 $\lceil \log_m n(m-1) + 1 \rceil \leq h \leq n - m + 1$

二叉树

- n : 二叉树节点数
- n_0 : 度为 0 的节点个数/叶节点个数
- n_2 : 度为 2 的节点个数/分支节点个数
- h : 树的高度

1. 非空二叉树:

$$n_0 = n_2 + 1, n = n_0 + n_1 + n_2$$

第 k 层最多有 2^{k-1} 个节点

整个树一共最多有 $2^h - 1$ 个节点

2. 完全二叉树:

$$h = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$$

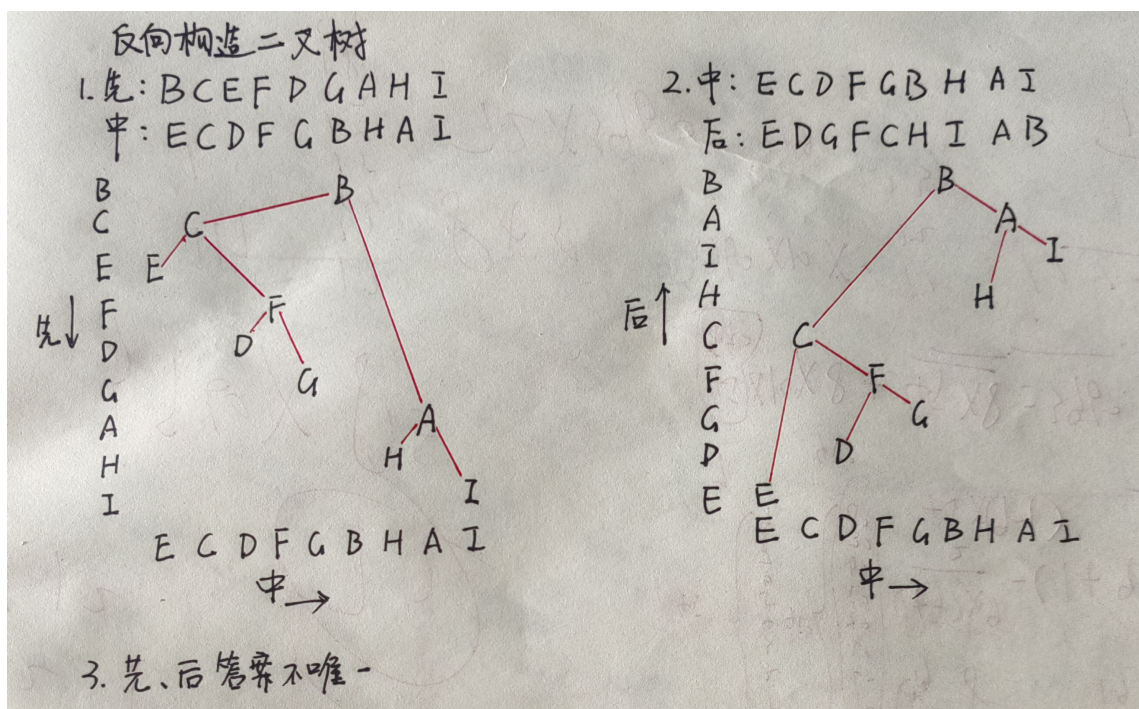
3. 有 n 个节点的二叉树采用链式存储, 空指针数为 $n+1$

二叉树的遍历

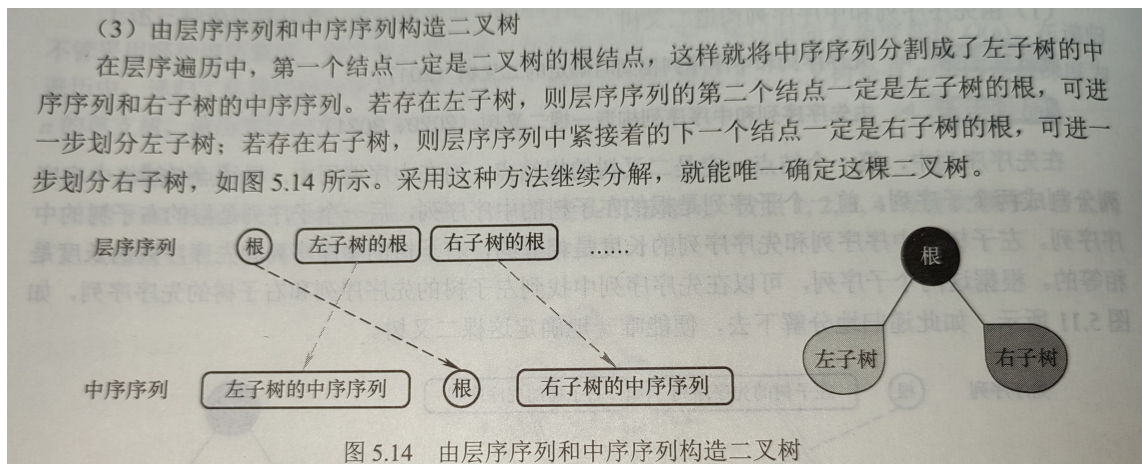
1. 已知先序、中序构造二叉树; 已知后序、中序构造二叉树

注:

1. 连线的时候不能跨节点 (对上面的节点做一条铅垂线, 下面的连线不能跨过该铅垂线)
2. 中序写下面写上面无所谓
3. 构造完成后必须检查一遍, 检验遍历顺序是否符合题目要求
4. 如果实在不能使用这个方法, 那先序就从头看起, 二分中序序列; 后序从尾看起, 二分中序序列



2. 已知层序、中序构造二叉树



请读者分析层序序列 (ABDCEFGIH) 和中序序列 (BCAEDGHI) 所确定的二叉树。

注意，先序序列、后序序列和层序序列的两两组合，无法唯一确定一棵二叉树。例如，图 5.15 所示的两棵二叉树的先序序列都为 AB，后序序列都为 BA，层序序列都为 AB。

3. 先序、后序和层序两两组合无法唯一确定一棵二叉树

4. 给定先序和后序序列，如何确定符合条件的二叉树数量

- 前序序列的首元素和后序序列的末元素是根节点，必须相同
- 遍历前序序列，对于每个节点 x (除最后一个)，找到其后一个节点 y
- 在后序序列中找到 x 的位置，其前一个节点若等于 y ，则说明 x 只有一个子节点 (左或右)，产生结构歧义
- 每出现一次这种情况，可能的树数量翻倍
- 故可能的二叉树数量为 2^k ，其中 k 是满足前序中某节点的后继等于后序中该节点的前驱的节点个数
- eg：前序 ABC 后序 CBA，A 和 B 满足条件，个数为 $2^k = 2^2 = 4$ 种
- 这个方法也可以用来确定二叉树的结构以及子节点

5. 二叉树是逻辑结构，线索二叉树是物理结构

6. 二叉树线索化后，空链域的个数为 2

7. 后序线索二叉树不能有效解决求后序后继的问题，故遍历后序线索二叉树需要栈的支持

8. 给定长度为 n 的先序序列求可能的二叉树个数

- 问题等同于给定长度为 n 的入栈序列，有多少种出栈序列
- 个数为 $\frac{1}{n+1} C_{2n}^n$

9. 非空二叉树先序序列和后序序列正好相反，二叉树的形态为：高度等于节点数

非空二叉树先序序列和后序序列正好相同，二叉树的形态为：只有一个根节点

10. 结点的带权路径长度 (WPL)：从树的根结点到任意结点的路径长度 (经过的边数) 与该结点上权值的乘积。

- 实际上是哈夫曼树的知识

树、森林

1. 森林的先根遍历序列对应于其二叉树的先序遍历序列，森林的后根遍历序列对应于其二叉树的中序遍历序列

树与二叉树的应用

1. 哈夫曼树中只有度为 0 和 2 的节点，节点总数为 $n = n_0 + n_2$ ，且 $n_0 = n_2 + 1$
 - 哈夫曼树总码字数量为叶子节点个数，即 n_0
 - 显然 n 一定不为偶数，即哈夫曼树总节点数不能是偶数
 - 分支数 = 边数 = $n - 1$
 - 哈夫曼树中只有度为 0 和 m 的节点，分支数 = $m * n_m = n - 1 = n_0 + n_m - 1$

第六章 图

图的基本概念

1. 有向图中每条有向边都有一个起点和终点，可推出
 - 顶点入度之和 = 顶点出度之和 = 边数
2. 子图：可能少边，可能少点；生成子图：少边，不少点
3. 非连通图边最多的情况：由 $n - 1$ 个顶点构成一个完全图，此时再加入一个顶点则变成非连通图
4. 有向图强连通情况下边最少的情况：至少需要 n 条边，构成一条环路
5. 完全图（也称简单完全图）
 - 无向图：
 $|E|$ 的取值范围为 0 到 $\frac{n(n-1)}{2}$ 。有 $\frac{n(n-1)}{2}$ 条边的无向图称为 **完全图**，在完全图中任意两个顶点之间都存在边
 - 有向图：
 $|E|$ 的取值范围为 0 到 $n(n-1)$ 。有 $n(n-1)$ 条弧的有向图称为 **有向完全图**，在有向完全图中任意两个顶点之间都存在方向相反的两条弧
6. 稠密图、稀疏图
 - 稀疏图：边数很少的图
 - 稠密图：边数较多的图
 - 稀疏和稠密是相对概念，一般当图 G 满足 $|E| < \sqrt{\log_2 |V|}$ 时，可视为稀疏图
7. 有向树
 - 若一个 **有向图** 满足：
 - 存在且仅存在一个顶点入度为 0（称为根节点），
 - 其余所有顶点的入度均为 1
 - 则称该有向图为 **有向树**
8. 无向图有 n 个顶点， m 条边
 - 顶点 V 的度： $TD(V)$
 - $\sum_{i=1}^n TD(V_i) = 2 * m$
9. 插入边使得无向图连通分量最多：由于插入一条边，连通分量个数 -1，尽可能将边插入一个连通分量中，使之变成完全图
 - 例如，具有 51 个顶点和 21 条边的无向图的连通分量最多为：7 个顶点的完全图有 21 条边，故连通分量个数为 $51 - 7 + 1 = 45$
10. 节点数为 n 的树，边数为 $n - 1$
11. n 个顶点的完全图，边数为 $\frac{n(n-1)}{2}$

- 要使 n 个顶点的无向图在任何情况下都是联通的，至少需要 $n - 1$ 个顶点构成完全图，再加一条边连接最后一个顶点，即 $\frac{(n-1)(n-2)}{2} + 1$

图的存储及基本操作

- 1. 十字链表与邻接多重表：BV1gqAZeyEaB
- 2. 设图 G 的邻接矩阵为 A ， A^n 的元素 a_{ij}^n 等于从顶点 i 到 j 的长度为 n 的路径的数目
 - 求 A^n
 - 对于无向图，是对称阵
 - 特别地，求 A^2 时，主对角线上的元素为对应节点的度
 - 数长度为 n 的路径条数即可

图的应用

最小生成树

- 1. 最小生成树是生成子图，具有树的性质，即边数为顶点数 - 1
- 2. 最小生成树不保证任意两个顶点之间的路径是最短路径
- 3. 唯一性总结
 - **MST 唯一的充要条件**：图中所有边的权重互不相同，且对于任意不在 MST 中的边 (u, v) ，其权重严格大于 MST 中 u 到 v 路径上所有边的权重
 - 边数 = $n - 1$ 时 MST 一定唯一；边数 > $n - 1$ 时 MST 可能唯一也可能不唯一

dijkstra

- 1. 算法流程
 - 初始化 dis ，若源点 s 到顶点 i 有边，就初始化为边的权值，否则初始化为 ∞
 - 初始化顶点集 S 只包含源点 s
 - 每次选择一个到顶点 s 距离最近的顶点 u 加入顶点集 S ，并判断由源点 s 绕行顶点 u 后到任一顶点 v 是否距离更短，若距离更短则将 dis_v 更新为 $dis_u + W_{uv}$ ，重复这个过程，直到所有顶点都加入顶点集 S

拓扑排序

- 1. 拓扑序列包含有向无环图的所有顶点

AOV 与 AOE 网对比

特征	AOV 网 (Activity On Vertex)	AOE 网 (Activity On Edge)
顶点含义	表示 活动	表示 事件 (活动的开始/结束点)
边含义	表示活动间的 依赖关系 (无权值)	表示 活动 (带权值，记录持续时间)
关注目标	活动的 执行顺序 (拓扑排序)	活动的 时间开销 (关键路径计算)
权值位置	无边权	边有权值 (活动耗时)
核心应用	解决任务调度问题	计算工程最短工期、关键活动
典型问题	能否有序完成所有活动?	完成工程的最短时间是多少?
结构示例	顶点：任务；边：任务A→任务B	边：任务 (权值=耗时)；顶点：事件

关键路径

1. 存在多条关键路径，当且仅当它们长度相等；延长任一关键活动必然延长工期；但缩短任一关键活动不一定缩短工期

第七章 查找

平均查找长度 ASL

1. 衡量查找算法效率的最重要的指标 $ASL = \sum_{i=1}^n P_i C_i$
- P_i : 查找第 i 个数据元素的概率，一般认为 $P_i = \frac{1}{n}$
 - C_i : 找到第 i 个数据元素所需进行的比较次数

2.	查找算法	流程描述	$ASL_{success}$	ASL_{fail}	时间复杂度	注
	一般线性表的顺序查找	从前往后/从后往前找	$\frac{n+1}{2}$ (1)	$n + 1$	$O(n)$	
	有序线性表的顺序查找	从前往后找，若第 i 个元素小于 $target$ ，第 $i + 1$ 个元素大于 $target$ ，则直接失败	$\frac{n+1}{2}$	$\frac{n}{2} + \frac{n}{n+1}$ (2)	$O(n)$	相比一般线性表的顺序查找降低了 ASL_{fail}
	折半查找 (二分查找)	每次取中，维护左右边界	$\frac{n+1}{n} \log_2 (n + 1) - 1 \approx \log_2 (n + 1) - 1$ (3)	$\lceil \log_2 (n + 1) \rceil$ (4)	$O(\log_2 n)$	

- (1): 从后往前: $\sum_{i=1}^n P_i (n - i + 1) = \frac{n+1}{2}$; 从前往后: $\sum_{i=1}^n P_i i = \frac{n+1}{2}$
- (2): 有 $n + 1$ 个失败区间，令 q_j 为到达第 j 个失败节点的概率， l_j 为到达第 j 个失败节点的比较次数

$$q_j = \frac{1}{n + 1}$$
$$l_j = \begin{cases} j & j = 1, 2, \dots, n \\ n & j = n + 1 \end{cases}$$
$$ASL_{fail} = \sum_{j=1}^{n+1} q_j l_j = \frac{n(n + 3)}{2(n + 1)} = \frac{n}{2} + \frac{n}{n + 1} \sim \frac{n}{2}$$

- (3)/(4): 二分查找的平均查找长度推导
 - 二分查找的判定树为一棵 AVL 树

■	名称	含义
	n	序列长度
	h	树高，从 1 开始
	内部节点	判定树中的非叶子结点，代表序列中的元素

名称	含义
外部节点	判定树中的叶子节点，代表失败区间
t	第 $h - 1$ 层内部节点数
s	第 $h - 1$ 层外部节点数
m	外部节点数
L_{h-1}	第 $h - 1$ 层外部节点数
L_h	第 h 层外部节点数
S	外部节点深度和

■ $ASL_{success}$ 推导：

- 有 $n = 2^h - 1$ ，于是树高 $h = \lceil \log_2(n + 1) \rceil$ ，于是有

$$ASL_{success} = \frac{1}{n} \sum_{i=1}^h l_i = \frac{1}{n} \sum_{i=1}^h i \cdot 2^{i-1} = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

- l_i ：判定树第 i 层所有节点的总比较次数， i 从 1 开始，对应深度 $i - 1$ ，该层节点数为 2^{i-1} ，每个节点的比较次数为 i ，故 $l_i = i \cdot 2^{i-1}$
- 使用错位相减法求解和式即可

■ ASL_{fail} 推导：

由AVL树性质： $s = 2^{h-1}$

$$m = (s - t) + 2t = s + t = n + 1$$

$$\therefore L_{h-1} = s - t = 2^{h-1} - (m - 2^{h-1}) = 2^h - m$$

$$L_h = 2t = 2(m - 2^{h-1})$$

$$\therefore S = L_{h-1} \cdot (h - 1) + L_h \cdot h$$

$$= (2^h - m)(h - 1) + 2(m - 2^{h-1})h$$

$$= m(h + 1) - 2^h$$

$$\therefore ASL_{fail} = \frac{1}{m} \cdot S = \frac{m(h + 1) - 2^h}{m} = (h + 1) - \frac{2^h}{n + 1} \quad \text{其中 } h = \lceil \log_2(n + 1) \rceil$$

第八章 排序